

INSTITUTO POLITÉCNICO NACIONAL

Escuela Superior de Comercio y Administración Unidad Santo Tomás

PROYECTO FINAL: PUGNELA

Equipo 5 – Pugnela

Unidad de aprendizaje: Laboratorio Empresarial
Grupo: 6GM1

Periodo escolar: 2026/1

Integrantes:

- Escobar Hernández Edna
- González Torres Fernanda
- López Reyes Fernanda
- Moreno Ruiz Diana Yael

Introducción

Actualmente, las plataformas digitales representan una herramienta fundamental para la comercialización de productos y servicios. El crecimiento del comercio electrónico ha permitido que pequeñas y medianas empresas amplíen su alcance mediante sistemas que facilitan la interacción con los clientes, la gestión de productos y la realización de compras en línea. En este contexto, el desarrollo de aplicaciones web se ha convertido en una competencia importante dentro del ámbito de los negocios digitales, ya que permite integrar conocimientos de programación, bases de datos, diseño de interfaces y experiencia de usuario.

Con el propósito de aplicar los conocimientos adquiridos durante el curso, el Equipo 5 desarrolló el proyecto denominado **Pugnela**, una tienda en línea especializada en la venta de roles de canela y productos de repostería. El proyecto fue construido utilizando PHP, MySQL, HTML, CSS y el patrón de arquitectura MVC (Modelo-Vista-Controlador), lo que permitió organizar adecuadamente cada componente del sistema y facilitar su mantenimiento.

Durante el desarrollo del proyecto se implementaron diversas funcionalidades que fueron aumentando progresivamente la complejidad del sistema. Inicialmente se trabajó en la configuración del entorno de desarrollo y la conexión a la base de datos. Posteriormente se desarrolló el sistema de autenticación de usuarios, la visualización dinámica de productos, la implementación del carrito de compras, la actualización automática del inventario y, finalmente, mecanismos de seguridad que fortalecieron la protección de la información y la integridad de la aplicación.

Uno de los principales retos enfrentados durante la elaboración del proyecto fue la configuración del entorno de trabajo, particularmente la modificación de los puertos de Apache y MySQL para resolver conflictos de conexión. Asimismo, fue necesario realizar pruebas constantes para garantizar la correcta comunicación entre la aplicación y la base de datos, así como validar que cada nueva funcionalidad se integrara correctamente con los módulos previamente desarrollados.

Como resultado, se obtuvo una aplicación web funcional capaz de simular el proceso de compra de una tienda en línea real, permitiendo a los usuarios iniciar sesión, consultar productos, agregarlos a un carrito de compras, modificar cantidades, eliminar artículos, finalizar compras y verificar que las medidas de seguridad implementadas funcionaran correctamente.

Objetivo General

Desarrollar una aplicación web funcional para la gestión de una tienda en línea denominada Pugnela, utilizando PHP, MySQL y la arquitectura MVC, permitiendo la administración de usuarios, productos, carrito de compras y procesos de compra de manera organizada, segura y eficiente.

Objetivos Específicos

- Diseñar y configurar una base de datos que permita almacenar usuarios, productos y operaciones del carrito de compras.
- Implementar un sistema de autenticación para controlar el acceso de los usuarios al sistema.
- Desarrollar un catálogo dinámico de productos conectado a una base de datos.
- Implementar un carrito de compras con operaciones de agregar, actualizar y eliminar productos.
- Desarrollar un proceso de checkout que permita finalizar compras y actualizar automáticamente el inventario.
- Incorporar medidas de seguridad para proteger la información y validar los datos ingresados por los usuarios.
- Aplicar la arquitectura MVC para mantener una estructura organizada y escalable del proyecto.

Descripción del Proyecto

Pugnela es una aplicación web desarrollada con el objetivo de simular el funcionamiento de una tienda en línea especializada en la venta de roles de canela y productos de repostería. La plataforma permite a los usuarios autenticarse mediante un sistema de inicio de sesión, visualizar productos almacenados en una base de datos, agregarlos a un carrito de compras y completar el proceso de compra.

El proyecto fue desarrollado siguiendo la arquitectura Modelo-Vista-Controlador (MVC), lo que permitió separar la lógica de negocio, la presentación visual y la gestión de datos. Gracias a esta estructura, cada módulo pudo desarrollarse de forma independiente y posteriormente integrarse en una solución completa.

Además de las funcionalidades principales de comercio electrónico, el sistema incorpora validaciones de seguridad que ayudan a prevenir errores y ataques comunes en aplicaciones web, fortaleciendo así la confiabilidad de la plataforma.

Tecnologías Utilizadas

Para el desarrollo del proyecto se utilizaron las siguientes tecnologías:

PHP

Lenguaje de programación utilizado para implementar la lógica del sistema, controlar el flujo de información y gestionar la interacción entre la aplicación y la base de datos.

MySQL

Sistema gestor de bases de datos utilizado para almacenar la información relacionada con usuarios, productos y operaciones del carrito de compras.

XAMPP

Herramienta utilizada para crear el entorno local de desarrollo, permitiendo ejecutar Apache y MySQL desde un mismo panel de control.

HTML

Lenguaje utilizado para estructurar las vistas de la aplicación.

CSS

Utilizado para personalizar la apariencia visual de la plataforma mediante una identidad gráfica inspirada en una pastelería.

Arquitectura MVC

Patrón de diseño empleado para separar responsabilidades dentro del sistema y mejorar su organización.

Arquitectura MVC Implementada

La aplicación fue desarrollada utilizando el patrón Modelo-Vista-Controlador (MVC), el cual permite dividir el sistema en tres componentes principales.

Modelo

El modelo es responsable de la gestión de datos y la comunicación con la base de datos.

Archivos principales:

- User.php
- Product.php
- Cart.php

Sus funciones incluyen:

- Consultar usuarios.

- Obtener productos.
- Gestionar el carrito de compras.
- Actualizar inventario.

Vista

Las vistas son las interfaces que interactúan directamente con el usuario.

Archivos principales:

- login.php
- dashboard/index.php
- cart/index.php

Sus funciones incluyen:

- Mostrar formularios.
- Visualizar productos.
- Mostrar el carrito de compras.
- Presentar mensajes al usuario.

Controlador

Los controladores actúan como intermediarios entre las vistas y los modelos.

Archivos principales:

- AuthController.php
- ProductController.php
- CartController.php

Sus funciones incluyen:

- Procesar solicitudes.
- Validar información.
- Gestionar sesiones.
- Coordinar las operaciones del sistema.

Diseño y Configuración de la Base de Datos

La base de datos utilizada para el proyecto fue denominada **pugnela_db**. Su diseño se realizó con el objetivo de almacenar de manera organizada toda la información necesaria para el funcionamiento de la aplicación.

Para evitar conflictos con otros servicios instalados en el equipo, fue necesario modificar los puertos predeterminados de Apache y MySQL.

Configuración utilizada

Apache:

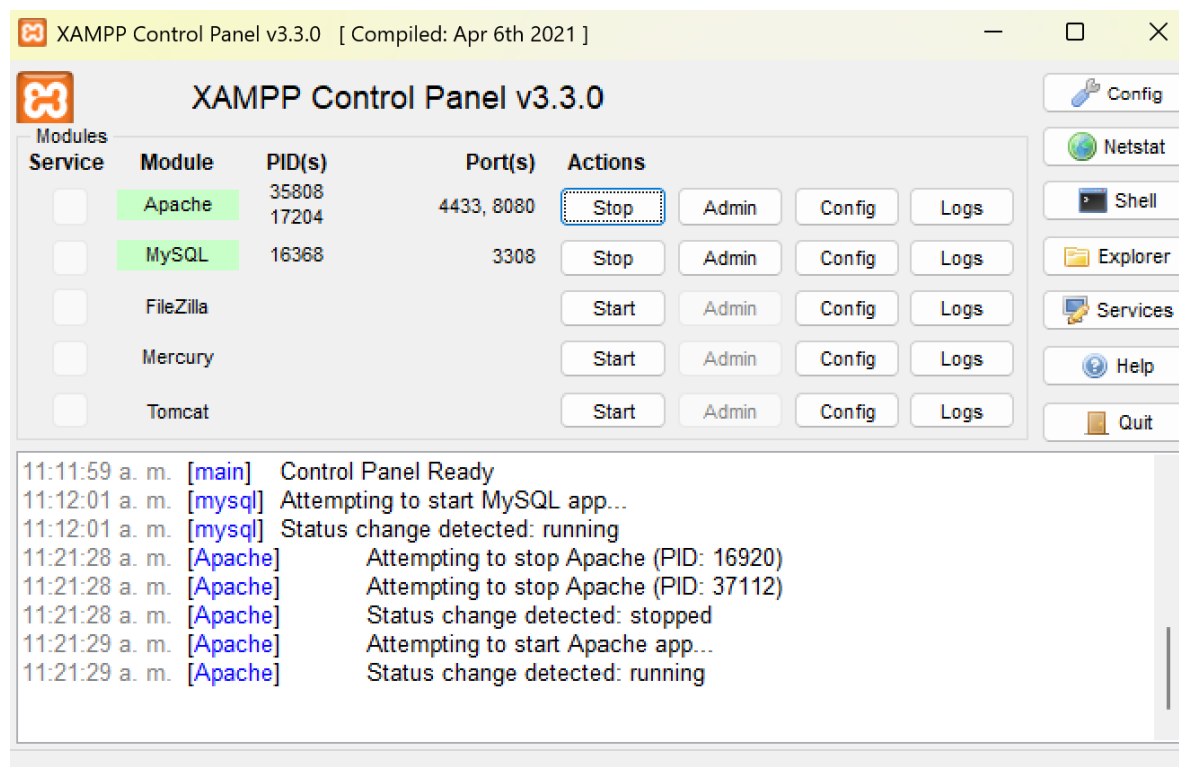
- Puerto original: 80
- Puerto configurado: 8080

MySQL:

- Puerto original: 3306
- Puerto configurado: 3308

Posteriormente se creó la base de datos y se definieron las tablas principales para almacenar usuarios, productos y elementos del carrito de compras.

Figura 1. Configuración inicial de la base de datos y entorno de desarrollo.



Descripción:

En esta imagen se muestra la configuración inicial del entorno de desarrollo utilizado para el proyecto. Se observa la correcta ejecución de Apache y MySQL, así como la estructura inicial de la base de datos utilizada por el sistema.

Implementación del Sistema de Autenticación

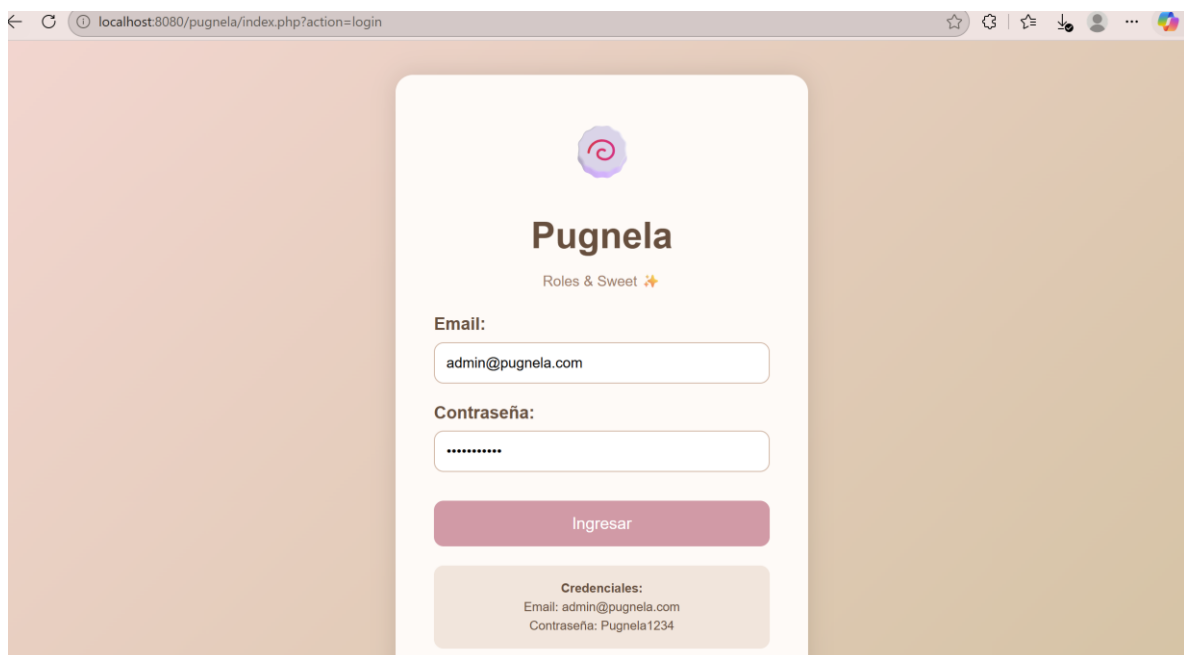
Una vez configurada la base de datos y establecida la conexión con MySQL, se procedió al desarrollo del sistema de autenticación. Esta funcionalidad representa uno de los componentes más importantes de la aplicación, ya que permite controlar el acceso a las diferentes secciones del sistema y garantizar que únicamente los usuarios autorizados puedan interactuar con la plataforma.

Para lograrlo se desarrolló el modelo **User.php**, encargado de consultar la información almacenada en la tabla usuarios y validar las credenciales proporcionadas durante el inicio de sesión. Asimismo, se implementó el controlador **AuthController.php**, responsable de procesar las solicitudes de autenticación, gestionar las sesiones y controlar la navegación entre las distintas vistas del sistema.

Adicionalmente, se diseñó una interfaz personalizada de inicio de sesión inspirada en la identidad visual de Pugnela, utilizando colores cálidos y elementos gráficos relacionados con la repostería para generar una experiencia visual más atractiva para el usuario.

Gracias a la implementación de este módulo, los usuarios pueden acceder al sistema mediante credenciales válidas, manteniendo protegidas las funcionalidades principales de la aplicación.

Figura 2. Pantalla de inicio de sesión de Pugnela.



Descripción:

La imagen muestra la interfaz principal de autenticación desarrollada para el sistema Pugnela. En ella se observan los campos de correo electrónico y contraseña necesarios para ingresar al sistema, así como la identidad visual diseñada específicamente para el proyecto.

Desarrollo del Catálogo de Productos

Después de implementar el sistema de autenticación, se desarrolló el catálogo de productos con el objetivo de mostrar de manera dinámica la información almacenada en la base de datos.

Para ello se creó el archivo **Product.php**, encargado de recuperar los registros de la tabla productos mediante consultas SQL. Posteriormente se desarrolló el controlador **ProductController.php**, el cual obtiene la información del modelo y la envía a la vista correspondiente.

El dashboard fue diseñado utilizando tarjetas dinámicas que muestran información relevante de cada producto, incluyendo nombre, precio, marca, stock disponible e imagen. Esta estructura permite que cualquier modificación realizada directamente en la base de datos se refleje automáticamente dentro del sistema sin necesidad de alterar el código de la interfaz.

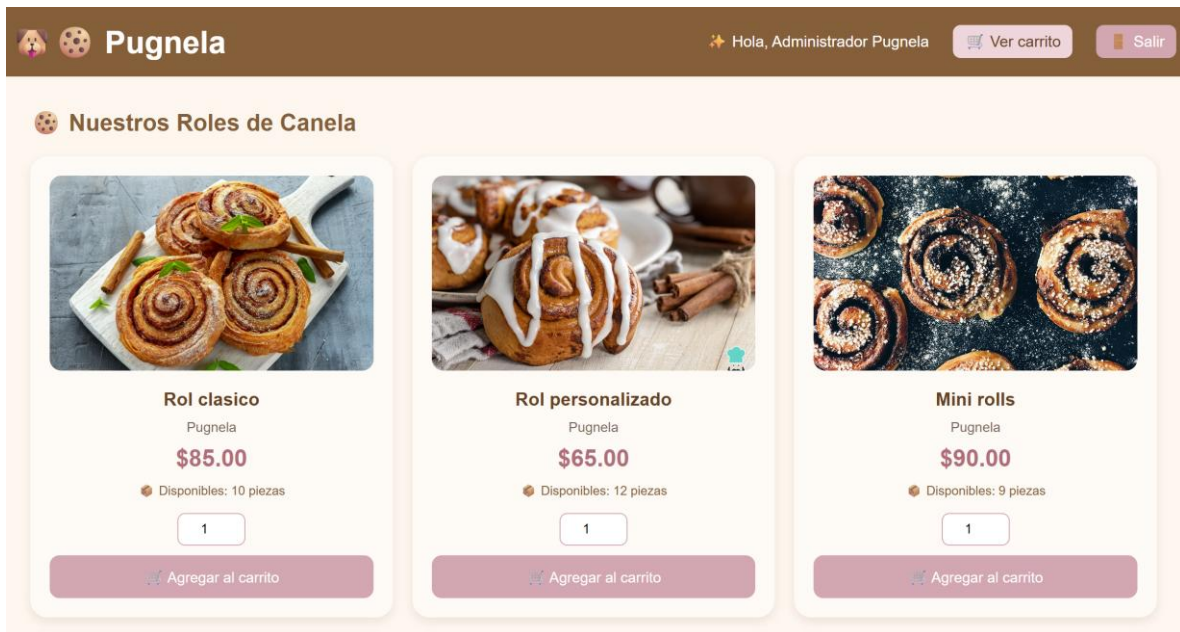
Además, fue necesario modificar la estructura de la tabla productos incorporando nuevos campos que enriquecieran la experiencia del usuario y facilitaran la administración del inventario.

Entre los campos agregados se encuentran:

- Marca.
- Stock.
- Imagen.
- Descripción.

La implementación de este módulo permitió transformar una simple consulta de base de datos en una interfaz visual interactiva capaz de presentar información de forma organizada y atractiva.

Figura 3. Dashboard principal de productos.



Descripción:

En esta imagen se observa el dashboard principal de Pugnela. Se muestran los diferentes productos disponibles para la venta junto con su precio, stock disponible e imagen correspondiente. Esta sección constituye el catálogo principal del sistema.

Implementación del Carrito de Compras

Una vez desarrollado el catálogo de productos, se procedió a implementar el carrito de compras, una funcionalidad fundamental dentro de cualquier plataforma de comercio electrónico.

Para ello se desarrolló el modelo **Cart.php**, encargado de gestionar las operaciones relacionadas con los productos seleccionados por el usuario. Entre sus principales funciones se encuentran la inserción de productos al carrito, la consulta de artículos almacenados, el cálculo automático del total y la eliminación completa de los registros cuando una compra es finalizada.

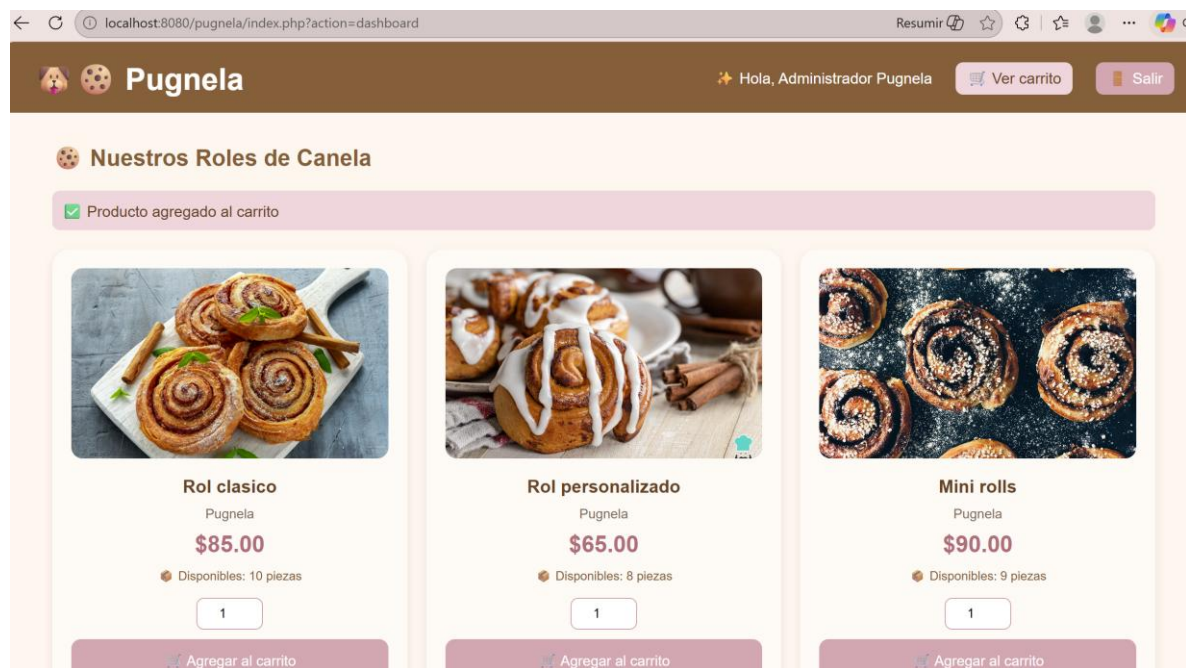
El desarrollo de este módulo permitió establecer una conexión directa entre los productos mostrados en el dashboard y la experiencia de compra del usuario.

Posteriormente se creó el controlador **CartController.php**, encargado de recibir las solicitudes provenientes de las vistas y comunicarse con el modelo correspondiente para ejecutar cada operación.

Además, fue necesario actualizar el archivo principal del proyecto (**index.php**) para incorporar las rutas correspondientes al carrito de compras y permitir la correcta navegación dentro del sistema.

Como resultado, los usuarios pueden seleccionar productos desde el catálogo principal y agregarlos a una lista temporal de compra que permanece disponible durante toda la sesión.

Figura 4. Producto agregado al carrito.



Descripción:

La imagen muestra el momento en que un producto es agregado exitosamente al carrito de compras. El sistema confirma la operación mediante un mensaje de éxito y almacena la información correspondiente dentro de la base de datos.

Figura 5. Vista general del carrito de compras.

Carrito Pugnela				
			Productos	Salir
Producto	Precio	Cantidad	Subtotal	Acción
Rol clasico	\$85.00	1	\$170.00	 Eliminar
Rol personalizado	\$65.00	2	\$130.00	 Eliminar
Total			\$300.00	
Actualizar Carrito				

Descripción:

En esta captura se observa el carrito de compras de Pugnela. Se muestran los productos seleccionados por el usuario, la cantidad elegida, el precio individual, los subtotales y el total general de la compra.

Gestión y Actualización del Carrito

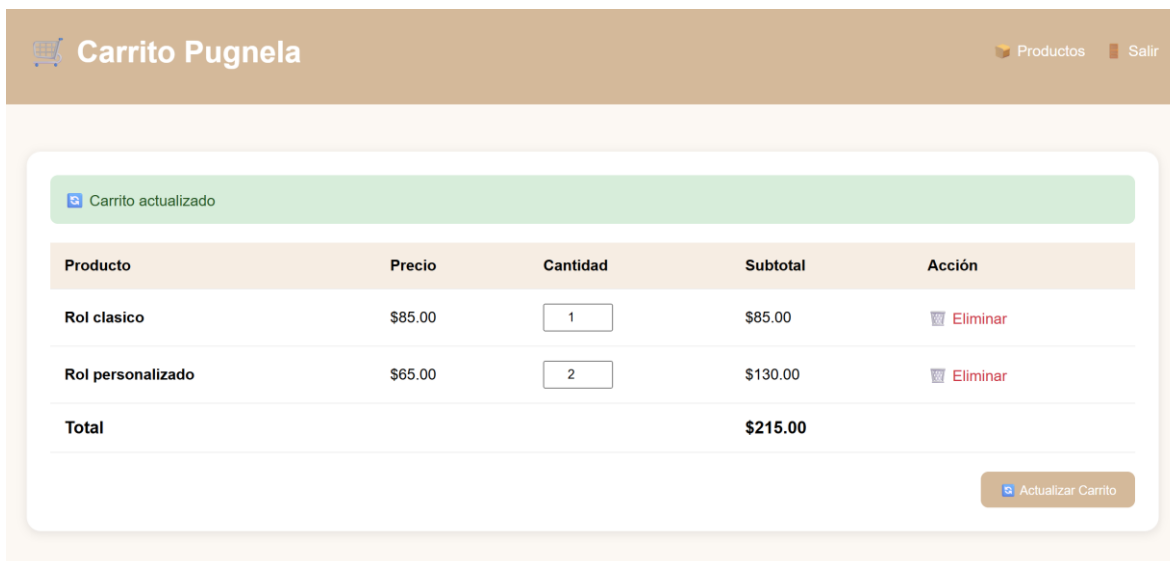
Con el propósito de mejorar la experiencia de compra, se incorporaron nuevas funcionalidades que permiten modificar la cantidad de productos agregados al carrito y eliminar artículos que ya no se desean adquirir.

Para ello se implementaron nuevos métodos dentro del modelo **Cart.php**, específicamente diseñados para actualizar cantidades y eliminar registros existentes. Estas operaciones trabajan directamente con la tabla carrito de la base de datos y permiten mantener la información actualizada en todo momento.

Asimismo, se desarrollaron nuevas funciones dentro del controlador **CartController.php**, responsables de procesar las solicitudes realizadas por el usuario desde la interfaz gráfica.

La integración de estas funciones permitió que el sistema recalculara automáticamente los subtotales y el total general después de cada modificación, garantizando la consistencia de la información mostrada.

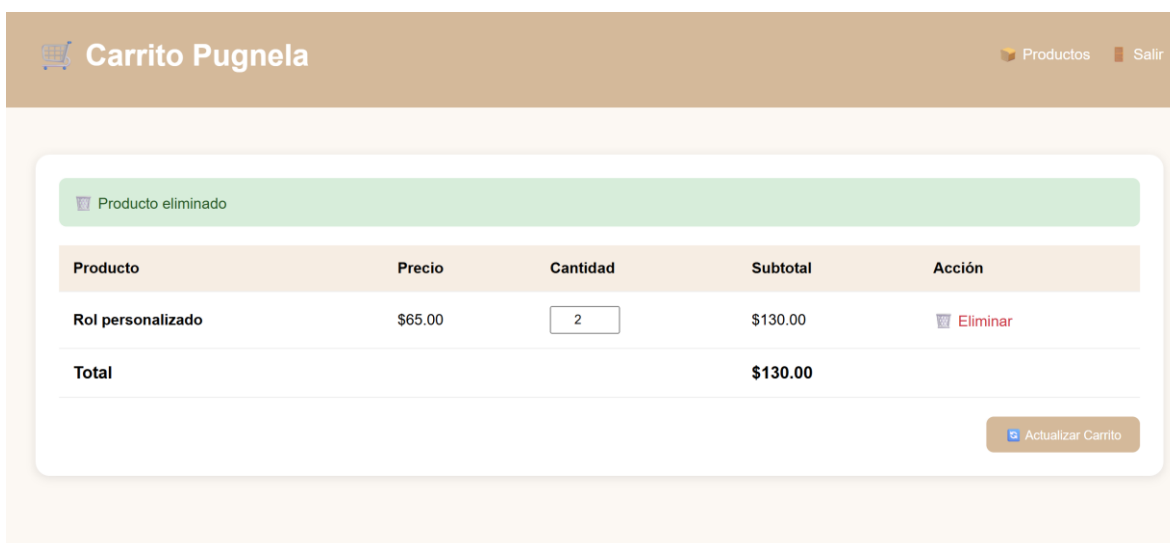
Figura 6. Actualización de cantidades dentro del carrito.



Descripción:

La imagen muestra la modificación de las cantidades seleccionadas por el usuario. Al actualizar los valores, el sistema recalcula automáticamente el importe total de la compra.

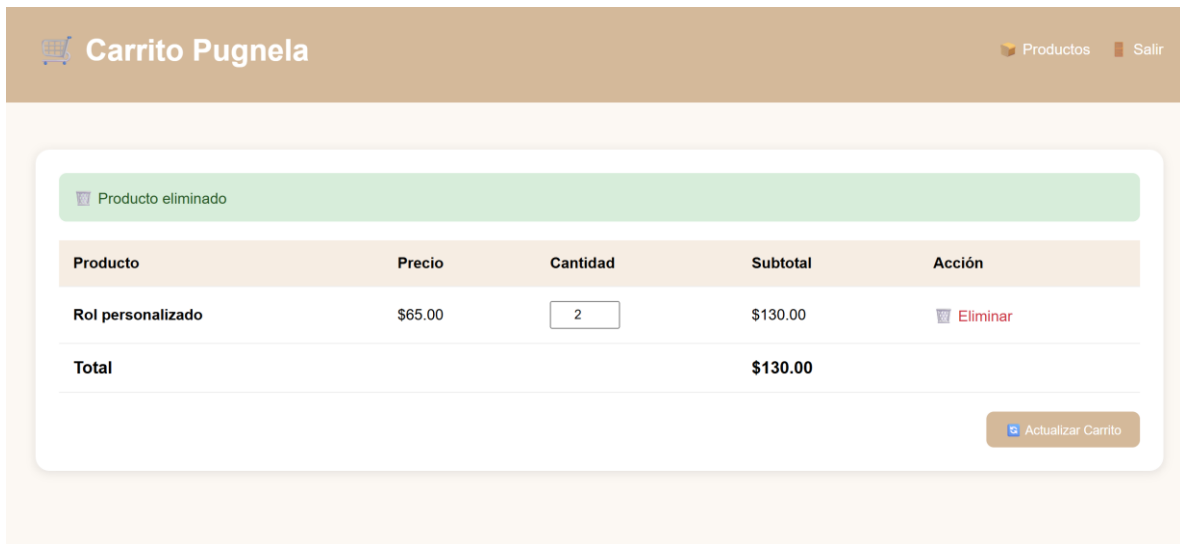
Figura 7. Eliminación de productos del carrito.



Descripción:

En esta captura se observa la opción que permite eliminar productos específicos del carrito mediante una acción directa del usuario.

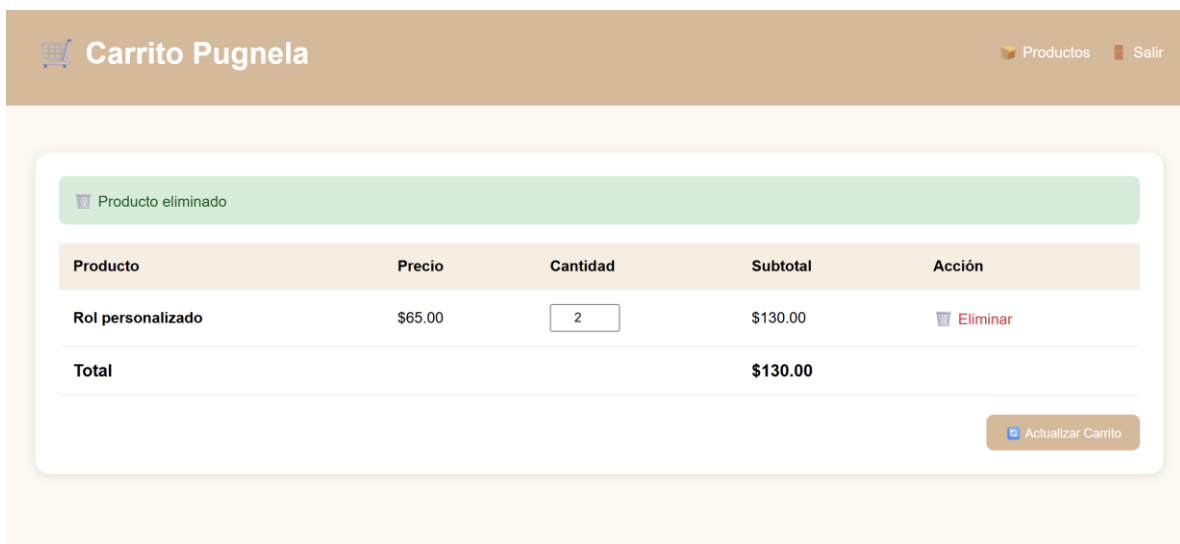
Figura 8. Confirmación de producto eliminado.



Descripción:

La imagen muestra el mensaje generado por el sistema después de eliminar correctamente un producto del carrito de compras.

Figura 9. Carrito actualizado después de la eliminación.



Descripción:

Se observa el carrito después de eliminar un producto. El sistema actualiza automáticamente los subtotales y el total general, mostrando únicamente los artículos restantes.

Implementación del Proceso de Compra

Después de desarrollar el carrito de compras y las funcionalidades de actualización y eliminación de productos, se implementó el proceso de checkout, el cual representa la etapa final del flujo de compra dentro de la plataforma Pugnela.

El objetivo principal de esta funcionalidad fue permitir que los usuarios completaran una compra de forma segura, garantizando la correcta validación del inventario disponible antes de confirmar la operación.

Para ello se incorporó el método **updateStock()** dentro del modelo Product.php, encargado de disminuir automáticamente la cantidad disponible de cada producto después de una compra exitosa.

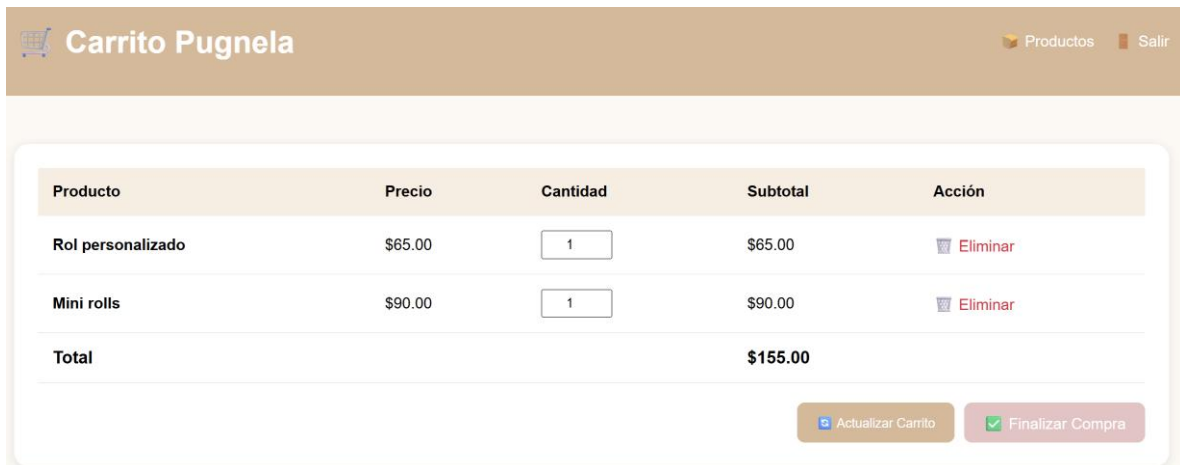
Posteriormente se desarrolló el método **checkout()** dentro del controlador CartController.php. Este procedimiento realiza diversas verificaciones antes de finalizar la compra:

- Verifica que el carrito no se encuentre vacío.
- Comprueba que los productos existan en la base de datos.
- Valida que exista stock suficiente para cada artículo.
- Actualiza automáticamente el inventario.
- Vacía el carrito después de completar la compra.
- Genera un mensaje de confirmación para el usuario.

Asimismo, fue necesario modificar el archivo principal del proyecto (index.php) para registrar la nueva ruta correspondiente al checkout e incorporar un botón dentro de la interfaz del carrito para ejecutar esta funcionalidad.

Gracias a esta implementación, el sistema es capaz de simular el proceso de compra de una tienda en línea real, manteniendo la consistencia de la información almacenada en la base de datos.

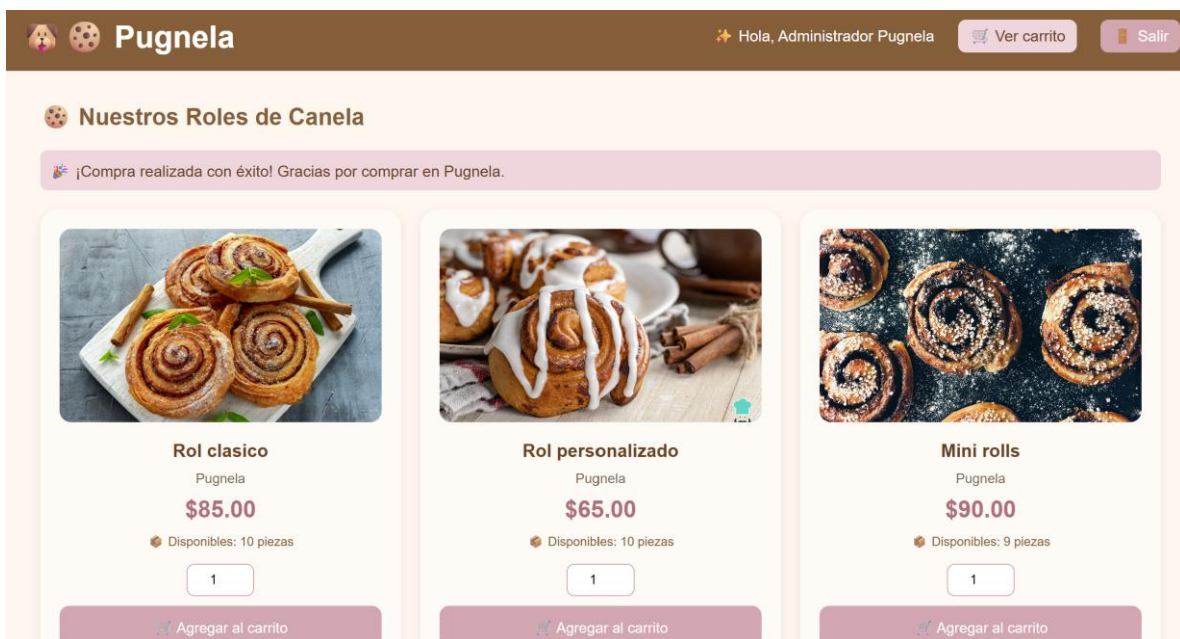
Figura 10. Botón “Finalizar Compra”.



Descripción:

La imagen muestra la interfaz del carrito de compras después de incorporar el botón “Finalizar Compra”. Esta opción permite iniciar el proceso de checkout y concluir la operación de compra.

Figura 11. Compra realizada exitosamente.



Descripción:

En esta captura se observa el mensaje generado por el sistema después de completar exitosamente una compra. El carrito es vaciado automáticamente y el usuario es redirigido al dashboard principal.

Implementación de Medidas de Seguridad

Una vez completadas las funcionalidades principales del sistema, se procedió a fortalecer la seguridad de la aplicación mediante la incorporación de mecanismos destinados a proteger la información almacenada y prevenir vulnerabilidades comunes en aplicaciones web.

La primera medida implementada fue la protección contra ataques CSRF (Cross-Site Request Forgery). Para ello se generó un token único durante la sesión del usuario y posteriormente se validó en cada solicitud enviada desde el formulario de autenticación.

Además, se incorporaron procesos de sanitización y validación de datos con el objetivo de evitar el ingreso de información maliciosa o incorrecta.

Dentro del módulo del carrito de compras también se añadieron verificaciones para impedir el registro de cantidades negativas, identificadores inválidos o solicitudes manipuladas.

Por otro lado, se implementó la regeneración automática de sesiones después del inicio de sesión exitoso, reduciendo el riesgo de secuestro de sesión.

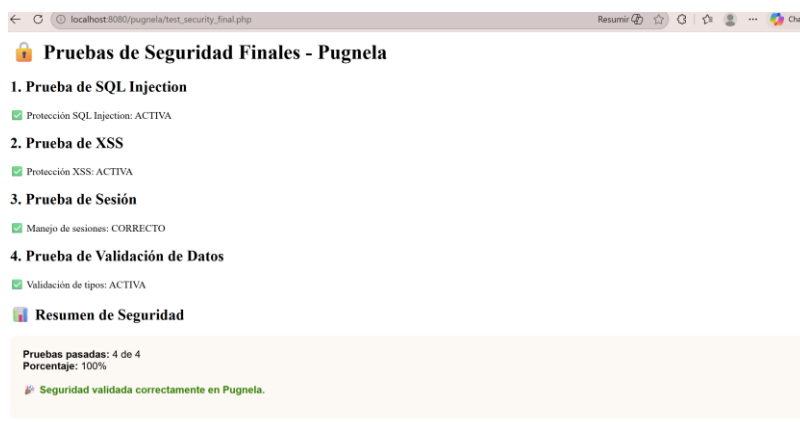
Finalmente, se desarrolló el archivo **test_security_final.php**, encargado de ejecutar diversas pruebas automatizadas para verificar el correcto funcionamiento de las medidas de seguridad implementadas.

Las pruebas realizadas incluyeron:

- SQL Injection.
- XSS (Cross Site Scripting).
- Manejo de sesiones.
- Validación de tipos de datos.

Todas las pruebas fueron ejecutadas exitosamente, obteniendo un porcentaje de cumplimiento del 100%.

Figura 12. Resultados de las pruebas de seguridad.



Descripción:

La imagen muestra la ejecución del archivo test_security_final.php. Se observa que las cuatro pruebas implementadas fueron aprobadas satisfactoriamente, validando el correcto funcionamiento de los mecanismos de seguridad incorporados en el sistema.

Resultados Obtenidos

Al finalizar el desarrollo del proyecto se logró construir una aplicación web completamente funcional para la gestión de una tienda en línea.

Entre los principales resultados obtenidos destacan los siguientes:

- Implementación de una base de datos relacional utilizando MySQL.
- Desarrollo de un sistema de autenticación basado en sesiones.
- Creación de un catálogo dinámico de productos conectado a la base de datos.
- Implementación de un carrito de compras funcional.
- Desarrollo de operaciones CRUD para la gestión de productos dentro del carrito.
- Integración de un sistema de checkout con actualización automática de inventario.
- Aplicación de mecanismos de validación y seguridad.
- Implementación de la arquitectura MVC para organizar el proyecto.
- Desarrollo de interfaces visuales personalizadas para la marca Pugnela.

Gracias a estos resultados fue posible construir una solución integral que simula de manera adecuada el funcionamiento de una tienda en línea real.

Conclusiones

El desarrollo del proyecto Pugnela permitió aplicar de manera práctica los conocimientos adquiridos a lo largo del curso en áreas relacionadas con programación web, bases de datos, arquitectura de software y seguridad informática. A través de las distintas etapas del proyecto fue posible comprender la importancia de planificar adecuadamente la estructura de una aplicación antes de comenzar su implementación.

Uno de los aspectos más valiosos del proyecto fue la aplicación de la arquitectura Modelo-Vista-Controlador (MVC), ya que permitió mantener una organización clara del código y facilitó la integración de nuevas funcionalidades conforme avanzaba el desarrollo. Gracias a esta estructura fue posible implementar de manera ordenada módulos como la autenticación de usuarios, el catálogo de productos, el carrito de compras y el proceso de checkout.

Durante la elaboración del proyecto también se enfrentaron diversos retos técnicos relacionados con la configuración del entorno de desarrollo, la conexión entre PHP y MySQL y la integración de nuevas funciones dentro del sistema. Sin embargo, mediante pruebas constantes y trabajo colaborativo fue posible resolver cada uno de estos problemas y garantizar el correcto funcionamiento de la aplicación.

Asimismo, la incorporación de mecanismos de seguridad permitió fortalecer la protección de la información y demostrar la importancia de implementar buenas prácticas durante el desarrollo de software. Las pruebas realizadas confirmaron que las medidas de validación y protección implementadas funcionan correctamente y contribuyen a la confiabilidad del sistema.

Finalmente, el proyecto permitió desarrollar una aplicación web funcional que cumple con los objetivos planteados inicialmente y que integra los principales elementos presentes en una plataforma de comercio electrónico. Además de los conocimientos técnicos adquiridos, esta experiencia fortaleció habilidades relacionadas con la resolución de problemas, el trabajo en equipo, la organización de proyectos y la toma de decisiones durante el desarrollo de soluciones digitales.

Anexos

Anexo A. Código completo de Database.php

```
<?php
```

```
class Database {
```

```
    private $host = "127.0.0.1";
```

```
    private $db_name = "pugna_db";
```

```
    private $username = "root";
```

```
    private $password = "";
```

```
    private $port = "3308";
```

```
    public $conn;
```

```
    public function getConnection() {
```

```

$this->conn = null;

try{

    $this->conn = new PDO(
        "mysql:host=" . $this->host . ";port=" . $this->port . ";dbname=" . $this->db_name,
        $this->username,
        $this->password
    );

    $this->conn->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
    $this->conn->exec("set names utf8");

} catch(PDOException $exception) {

    echo "Error de conexión: " . $exception->getMessage();
}

return $this->conn;
}

?>

```

Anexo B. Código completo de User.php

```

<?php

require_once __DIR__ . '/../config/database.php';

```

```
class User{

    private $conn;

    private $table_name = "usuarios";

    public function __construct() {

        $database = new Database();

        $this->conn = $database->getConnection();

    }

    public function login($email, $password) {

        $query = "SELECT * FROM " . $this->table_name . " WHERE email = :email";

        $stmt = $this->conn->prepare($query);

        $stmt->bindParam(':email', $email);

        $stmt->execute();

        if($stmt->rowCount() > 0) {

            $user = $stmt->fetch(PDO::FETCH_ASSOC);

            if($password == $user['password']) {

                return $user;

            }

        }

    }

}
```

```

        return false;
    }

    public function getUserById($id) {

        $query = "SELECT id, nombre, email FROM " . $this->table_name . " WHERE id = :id";

        $stmt = $this->conn->prepare($query);

        $stmt->bindParam(':id', $id);

        $stmt->execute();

        return $stmt->fetch(PDO::FETCH_ASSOC);
    }
}

?>

```

Anexo C. Código completo de Product.php

```

<?php

require_once 'models/Product.php';

class ProductController {

    private $productModel;

```

```

public function __construct() {

    $this->productModel = new Product();
}

public function dashboard() {

    $products = $this->productModel->getAll();

    include 'views/dashboard/index.php';
}
}

?>

```

Anexo D. Código completo de Cart.php

```

<?php

require_once __DIR__ . '/../config/database.php';

class Product {

    private $conn;

    private $table_name = "productos";

    public function __construct() {

        $database = new Database();

        $this->conn = $database->getConnection();
    }
}

```

```
}
```

```
public function getAll() {
```

```
    $query = "SELECT * FROM " . $this->table_name . "  
        ORDER BY created_at DESC";
```

```
    $stmt = $this->conn->prepare($query);
```

```
    $stmt->execute();
```

```
    return $stmt->fetchAll(PDO::FETCH_ASSOC);
```

```
}
```

```
public function getById($id) {
```

```
    $query = "SELECT * FROM " . $this->table_name . "  
        WHERE id = :id";
```

```
    $stmt = $this->conn->prepare($query);
```

```
    $stmt->bindParam(':id', $id);
```

```
    $stmt->execute();
```

```
    return $stmt->fetch(PDO::FETCH_ASSOC);
```

```
}
```

```
public function updateStock($id, $cantidad) {
```

```

        $query = "UPDATE " . $this->table_name . "
            SET stock = stock - :cantidad
            WHERE id = :id
            AND stock >= :cantidad";

        $stmt = $this->conn->prepare($query);

        $stmt->bindParam(':id', $id);
        $stmt->bindParam(':cantidad', $cantidad);

        return $stmt->execute();
    }
}

?>

```

Anexo E. Código completo de AuthController.php

```

<?php

require_once 'config/database.php';

class Cart {

    private $conn;

    private $table_name = "carrito";

    public function __construct() {
        $database = new Database();
    }
}

```



```
$this->conn = $database->getConnection();  
}
```

```
public function addToCart($usuario_id, $producto_id, $cantidad = 1){
```

```
    $query = "SELECT * FROM " . $this->table_name . "  
        WHERE usuario_id = :usuario_id  
        AND producto_id = :producto_id";
```

```
    $stmt = $this->conn->prepare($query);  
    $stmt->bindParam(':usuario_id', $usuario_id);  
    $stmt->bindParam(':producto_id', $producto_id);  
    $stmt->execute();
```

```
    if($stmt->rowCount() > 0) {  
        $query = "UPDATE " . $this->table_name . "  
            SET cantidad = cantidad + :cantidad  
            WHERE usuario_id = :usuario_id  
            AND producto_id = :producto_id";  
    } else {  
        $query = "INSERT INTO " . $this->table_name . "  
            (usuario_id, producto_id, cantidad)  
            VALUES  
            (:usuario_id, :producto_id, :cantidad)";  
    }  
}
```

```
    $stmt = $this->conn->prepare($query);  
    $stmt->bindParam(':usuario_id', $usuario_id);  
    $stmt->bindParam(':producto_id', $producto_id);
```

```

$stmt->bindParam(':cantidad', $cantidad);

return $stmt->execute();
}

public function getCartItems($usuario_id) {

$query = "SELECT c.*, p.nombre, p.precio, p.imagen
        FROM " . $this->table_name . " c
        JOIN productos p
        ON c.producto_id = p.id
        WHERE c.usuario_id = :usuario_id";

$stmt = $this->conn->prepare($query);
$stmt->bindParam(':usuario_id', $usuario_id);
$stmt->execute();

return $stmt->fetchAll(PDO::FETCH_ASSOC);
}

public function getCartTotal($usuario_id) {

$items = $this->getCartItems($usuario_id);
$total = 0;

foreach($items as $item) {
    $total += $item['precio'] * $item['cantidad'];
}
}

```

```
    return $total;
}
```

```
public function updateQuantity($cart_id, $usuario_id, $cantidad) {
```

```
    $query = "UPDATE " . $this->table_name . "
```

```
        SET cantidad = :cantidad
```

```
        WHERE id = :cart_id
```

```
        AND usuario_id = :usuario_id";
```

```
    $stmt = $this->conn->prepare($query);
```

```
    $stmt->bindParam(':cantidad', $cantidad);
```

```
    $stmt->bindParam(':cart_id', $cart_id);
```

```
    $stmt->bindParam(':usuario_id', $usuario_id);
```

```
    return $stmt->execute();
```

```
}
```

```
public function removeFromCart($cart_id, $usuario_id) {
```

```
    $query = "DELETE FROM " . $this->table_name . "
```

```
        WHERE id = :cart_id
```

```
        AND usuario_id = :usuario_id";
```

```
    $stmt = $this->conn->prepare($query);
```

```
    $stmt->bindParam(':cart_id', $cart_id);
```

```
    $stmt->bindParam(':usuario_id', $usuario_id);
```

```
    return $stmt->execute();
```

```

    }

    public function clearCart($usuario_id) {

        $query = "DELETE FROM " . $this->table_name . "
            WHERE usuario_id = :usuario_id";

        $stmt = $this->conn->prepare($query);
        $stmt->bindParam(':usuario_id', $usuario_id);

        return $stmt->execute();
    }
}

?>

```

Anexo F. Código completo de ProductController.php

```

<?php

if(session_status() === PHP_SESSION_NONE) {
    session_start();
}

require_once __DIR__ . '/../models/User.php';

class AuthController{

    private $userModel;

```

```
public function __construct() {  
    $this->userModel = new User();  
}
```

```
public function showLogin() {  
    include __DIR__ . '/../views/auth/login.php';  
}
```

```
public function login() {
```

```
    if($_SERVER['REQUEST_METHOD'] == 'POST') {
```

```
        if(!isset($_POST['csrf_token']) || $_POST['csrf_token'] !== $_SESSION['csrf_token']) {  
            $_SESSION['error'] = "Error de seguridad. Intenta nuevamente.";  
            header('Location: index.php?action=login');  
            exit();  
        }
```

```
        $email = filter_var($_POST['email'], FILTER_SANITIZE_EMAIL);
```

```
        $password = $_POST['password'];
```

```
        $user = $this->userModel->login($email, $password);
```

```
        if($user) {
```

```
            session_regenerate_id(true);
```

```
            $_SESSION['user_id'] = $user['id'];
```

```
            $_SESSION['user_name'] = $user['nombre'];
```

```
$_SESSION['user_email'] = $user['email'];

header('Location: index.php?action=dashboard');

exit();

} else {

$_SESSION['error'] = "Email o contraseña incorrectos";

header('Location: index.php?action=login');

exit();

}

}

}
```

```
public function logout() {
```

```
    session_destroy();
```

```
    header('Location: index.php?action=login');
```

```
    exit();
```

```
}
```

```
public function checkAuth() {
```

```
    if(!isset($_SESSION['user_id'])) {
```

```
        header('Location: index.php?action=login');
```

```
        exit();
    }
}
}

?>
```

Anexo G. Código completo de CartController.php

```
<?php

require_once 'models/Cart.php';
require_once 'models/Product.php';

class CartController {

    private $cartModel;
    private $productModel;

    public function __construct() {
        $this->cartModel = new Cart();
        $this->productModel = new Product();
    }

    public function addToCart() {

        if($_SERVER['REQUEST_METHOD'] == 'POST') {

            $usuario_id = $_SESSION['user_id'];
```

```
$producto_id = (int)$_POST['product_id'];  
$cantidad = (int)($_POST['cantidad'] ?? 1);
```

```
if($producto_id <= 0 || $cantidad <= 0) {  
    $_SESSION['error'] = "❌ Datos inválidos";  
    header('Location: index.php?action=dashboard');  
    exit();  
}
```

```
$product = $this->productModel->getById($producto_id);
```

```
if(!$product) {  
    $_SESSION['error'] = "❌ Producto no encontrado";  
    header('Location: index.php?action=dashboard');  
    exit();  
}
```

```
if($cantidad > $product['stock']) {  
    $_SESSION['error'] = "❌ No hay suficiente stock. Stock disponible: " .  
$product['stock'];  
    header('Location: index.php?action=dashboard');  
    exit();  
}
```

```
if($this->cartModel->addToCart($usuario_id, $producto_id, $cantidad)) {  
    $_SESSION['success'] = "✅ Producto agregado al carrito";  
} else {  
    $_SESSION['error'] = "❌ Error al agregar producto";  
}
```



```
        header('Location: index.php?action=dashboard');
        exit();
    }
}
```

```
public function viewCart() {
```

```
    $usuario_id = $_SESSION['user_id'];
```

```
    $cartItems = $this->cartModel->getCartItems($usuario_id);
```

```
    $total = $this->cartModel->getCartTotal($usuario_id);
```

```
    include 'views/cart/index.php';
}
```

```
public function removeFromCart() {
```

```
    if(isset($_GET['id'])) {
```

```
        $usuario_id = $_SESSION['user_id'];
```

```
        $cart_id = (int)$_GET['id'];
```

```
        if($cart_id <= 0) {
```

```
            $_SESSION['error'] = "❌ Producto inválido";
```

```
            header('Location: index.php?action=cart');
```

```
            exit();
```

```
        }
```

```

$this->cartModel->removeFromCart($cart_id, $usuario_id);

$_SESSION['success'] = " 🗑️ Producto eliminado";

header('Location: index.php?action=cart');
exit();
}
}

public function updateCart() {

if($_SERVER['REQUEST_METHOD'] == 'POST') {

    $usuario_id = $_SESSION['user_id'];

    if(!isset($_POST['quantities']) || empty($_POST['quantities'])) {
        $_SESSION['error'] = " ❌ No hay productos para actualizar";
        header('Location: index.php?action=cart');
        exit();
    }

    foreach($_POST['quantities'] as $cart_id => $cantidad) {

        $cart_id = (int)$cart_id;
        $cantidad = (int)$cantidad;

        if($cart_id > 0 && $cantidad > 0) {
            $this->cartModel->updateQuantity(

```

```
        $cart_id,  
        $usuario_id,  
        $cantidad  
    );  
}  
}
```

```
$_SESSION['success'] = "🛒 Carrito actualizado";
```

```
header('Location: index.php?action=cart');  
exit();  
}  
}
```

```
public function checkout() {
```

```
    $usuario_id = $_SESSION['user_id'];
```

```
    $cartItems = $this->cartModel->getCartItems($usuario_id);
```

```
    if(empty($cartItems)) {
```

```
        $_SESSION['error'] = "❌ Tu carrito está vacío";
```

```
        header('Location: index.php?action=cart');
```

```
        exit();
```

```
    }
```

```
    foreach($cartItems as $item) {
```

```
        $product = $this->productModel->getById($item['producto_id']);
```

```
if(!$product) {  
    $_SESSION['error'] = " ❌ Producto no encontrado";  
    header('Location: index.php?action=cart');  
    exit();  
}
```

```
if($product['stock'] < $item['cantidad']) {  
    $_SESSION['error'] = " ❌ Stock insuficiente para: " . $item['nombre'];  
    header('Location: index.php?action=cart');  
    exit();  
}  
}
```

```
foreach($cartItems as $item) {
```

```
    $this->productModel->updateStock(  
        $item['producto_id'],  
        $item['cantidad']  
    );  
}
```

```
$this->cartModel->clearCart($usuario_id);
```

```
$_SESSION['success'] = " 🎉 ¡Compra realizada con éxito! Gracias por comprar en  
Pugnela.";
```

```
header('Location: index.php?action=dashboard');  
exit();
```

```
}  
}
```

```
?>
```

Anexo H. Código completo de index.php

```
<?php
```

```
require_once 'models/Cart.php';
```

```
require_once 'models/Product.php';
```

```
class CartController {
```

```
    private $cartModel;
```

```
    private $productModel;
```

```
    public function __construct() {
```

```
        $this->cartModel = new Cart();
```

```
        $this->productModel = new Product();
```

```
    }
```

```
    public function addToCart() {
```

```
        if($_SERVER['REQUEST_METHOD'] == 'POST') {
```

```
            $usuario_id = $_SESSION['user_id'];
```

```
            $producto_id = (int)$_POST['product_id'];
```

```
            $cantidad = (int)($_POST['cantidad'] ?? 1);
```

```
if($producto_id <= 0 || $cantidad <= 0) {
```

```
    $_SESSION['error'] = "✖ Datos inválidos";
```

```
    header('Location: index.php?action=dashboard');
```

```
    exit();
```

```
}
```

```
$product = $this->productModel->getById($producto_id);
```

```
if(!$product) {
```

```
    $_SESSION['error'] = "✖ Producto no encontrado";
```

```
    header('Location: index.php?action=dashboard');
```

```
    exit();
```

```
}
```

```
if($cantidad > $product['stock']) {
```

```
    $_SESSION['error'] =
```

```
    "✖ No hay suficiente stock. Stock disponible: "
```

```
    . $product['stock'];
```

```
    header('Location: index.php?action=dashboard');
```

```
    exit();
```

```
}
```

```
if(
```

```

        $this->cartModel->addToCart(
            $usuario_id,
            $producto_id,
            $cantidad
        )
    ){

        $_SESSION['success'] =
            "✅ Producto agregado al carrito";

    } else {

        $_SESSION['error'] =
            "❌ Error al agregar producto";
    }

    header("Location: index.php?action=dashboard");
    exit();
}
}

```

```

public function viewCart() {

```

```

    $usuario_id = $_SESSION['user_id'];

```

```

    $cartItems =

```

```

    $this->cartModel->getCartItems($usuario_id);

```

```

    $total =

```

```
$this->cartModel->getCartTotal($usuario_id);
```

```
include 'views/cart/index.php';
```

```
}
```

```
public function removeFromCart() {
```

```
if(isset($_GET['id'])) {
```

```
    $usuario_id = $_SESSION['user_id'];
```

```
    $cart_id = (int)$_GET['id'];
```

```
    if($cart_id <= 0) {
```

```
        $_SESSION['error'] =
```

```
        "❌ Producto inválido";
```

```
        header('Location: index.php?action=cart');
```

```
        exit();
```

```
    }
```

```
$this->cartModel->removeFromCart(
```

```
    $cart_id,
```

```
    $usuario_id
```

```
);
```

```
$_SESSION['success'] =
```

```
"🗑️ Producto eliminado";
```



```
        header('Location: index.php?action=cart');  
        exit();  
    }  
}
```

```
public function updateCart() {
```

```
    if($_SERVER['REQUEST_METHOD'] == 'POST') {
```

```
        $usuario_id = $_SESSION['user_id'];
```

```
        if(  
            !isset($_POST['quantities']) ||  
            empty($_POST['quantities'])  
        ){
```

```
            $_SESSION['error'] =
```

```
            "✖ No hay productos para actualizar";
```

```
            header('Location: index.php?action=cart');  
            exit();  
        }
```

```
        foreach(  
            $_POST['quantities']  
            as $cart_id => $cantidad  
        ){
```

```
            $cart_id = (int)$cart_id;
```

```
$cantidad = (int)$cantidad;
```

```
if($cart_id > 0 && $cantidad > 0) {
```

```
    $this->cartModel->updateQuantity(
```

```
        $cart_id,
```

```
        $usuario_id,
```

```
        $cantidad
```

```
    );
```

```
}
```

```
}
```

```
$_SESSION['success'] =
```

```
" Carrito actualizado";
```

```
header('Location: index.php?action=cart');
```

```
exit();
```

```
}
```

```
}
```

```
public function checkout() {
```

```
    $usuario_id = $_SESSION['user_id'];
```

```
    $cartItems =
```

```
    $this->cartModel->getCartItems($usuario_id);
```

```
    if(empty($cartItems)) {
```

```
$_SESSION['error'] =
```

```
"✖ Tu carrito está vacío";
```

```
header('Location: index.php?action=cart');
```

```
exit();
```

```
}
```

```
foreach($cartItems as $item) {
```

```
    $product =
```

```
    $this->productModel->getById(
```

```
        $item['producto_id']
```

```
    );
```

```
    if(!$product) {
```

```
        $_SESSION['error'] =
```

```
        "✖ Producto no encontrado";
```

```
        header('Location: index.php?action=cart');
```

```
        exit();
```

```
    }
```

```
    if(
```

```
        $product['stock'] <
```

```
        $item['cantidad']
```

```
    ){
```

```
        $_SESSION['error'] =
```

```

        "❌ Stock insuficiente para: "
        . $item['nombre'];

        header('Location: index.php?action=cart');
        exit();
    }
}

foreach($cartItems as $item) {

    $this->productModel->updateStock(
        $item['producto_id'],
        $item['cantidad']
    );
}

$this->cartModel->clearCart($usuario_id);

$_SESSION['success'] =
    "🎉 ¡Compra realizada con éxito! Gracias por comprar en Pugnela.";

    header('Location: index.php?action=dashboard');
    exit();
}
}

?>

```

Anexo I. Código completo de test_security_final.php

```
<?php
```

```
echo "<h1>🔒 Pruebas de Seguridad Finales - Pugneta</h1>";
```

```
$tests = [];
```

```
echo "<h2>1. Prueba de SQL Injection</h2>";
```

```
$malicious = "' OR '1'='1";
```

```
require_once 'models/User.php';
```

```
$userModel = new User();
```

```
$result = $userModel->login($malicious, 'anything');
```

```
$tests['sql_injection'] = ($result === false);
```

```
echo ($tests['sql_injection'] ? "✅ " : "❌ ");
```

```
echo "Protección SQL Injection: ";
```

```
echo ($tests['sql_injection'] ? "ACTIVA" : "FALLA");
```

```
echo "<br>";
```

```
echo "<h2>2. Prueba de XSS</h2>";
```

```
$xss_test = "<script>alert('XSS')</script>";
```

```
$safe = htmlspecialchars($xss_test);
```

```
$tests['xss'] = ($safe !== $xss_test);
```

```
echo ($tests['xss'] ? "✅ " : "❌ ");
```

```
echo "Protección XSS: ";
```

```
echo ($tests['xss'] ? "ACTIVA" : "FALLA");
```

```
echo "<br>";
```

```
echo "<h2>3. Prueba de Sesión</h2>";
```

```
if(session_status() === PHP_SESSION_NONE) {
```

```
    session_start();
```

```
}
```

```
$session_id = session_id();
```

```
$tests['session'] = !empty($session_id);
```

```
echo ($tests['session'] ? "✅ " : "❌ ");
```

```
echo "Manejo de sesiones: ";
```

```
echo ($tests['session'] ? "CORRECTO" : "ERROR");
```

```
echo "<br>";
```

```
echo "<h2>4. Prueba de Validación de Datos</h2>";
```

```
$test_id = "abc123";
```

```
$clean_id = (int)$test_id;
```

```
$tests['validation'] = ($clean_id === 0);
```

```
echo ($tests['validation'] ? "✅ " : "❌ ");
echo "Validación de tipos: ";
echo ($tests['validation'] ? "ACTIVA" : "FALLA");
echo "<br>";

echo "<h2> 🏠 Resumen de Seguridad</h2>";

$passed = count(array_filter($tests));
$total = count($tests);

$percentage = ($passed / $total) * 100;

echo "<div style='background:#fdf8f3; padding:20px; border-radius:10px; font-family:Arial;'>";

echo "<strong>Pruebas pasadas:</strong> $passed de $total<br>";
echo "<strong>Porcentaje:</strong> $percentage%<br>";

if($percentage == 100) {

    echo "<p style='color:green; font-weight:bold;'> 🎉 Seguridad validada correctamente en Pugnela.</p>";

} else {

    echo "<p style='color:red; font-weight:bold;'> ⚠️ Revisar validaciones de seguridad.</p>";

}

echo "</div>";
```

?>